

Lecture 05: Exploration. N-step TD-learning

Petr Kuderov

Researcher at MIPT & AIRI

February 25, 2026

Exploration: Why Do We Need It?

We assume unknown action values.

Given action-value estimates $Q(s_t, a)$, greedy policy:

$$a_t = \arg \max_a Q(s_t, a)$$

If estimates are biased early, greedy policy may:

- lock onto suboptimal policy
- never correct the mistake

Exploration–exploitation dilemma:

- Exploit: maximize immediate expected reward.
- Explore: gather information to improve future return.

Bandits Setting

Long-term objective:

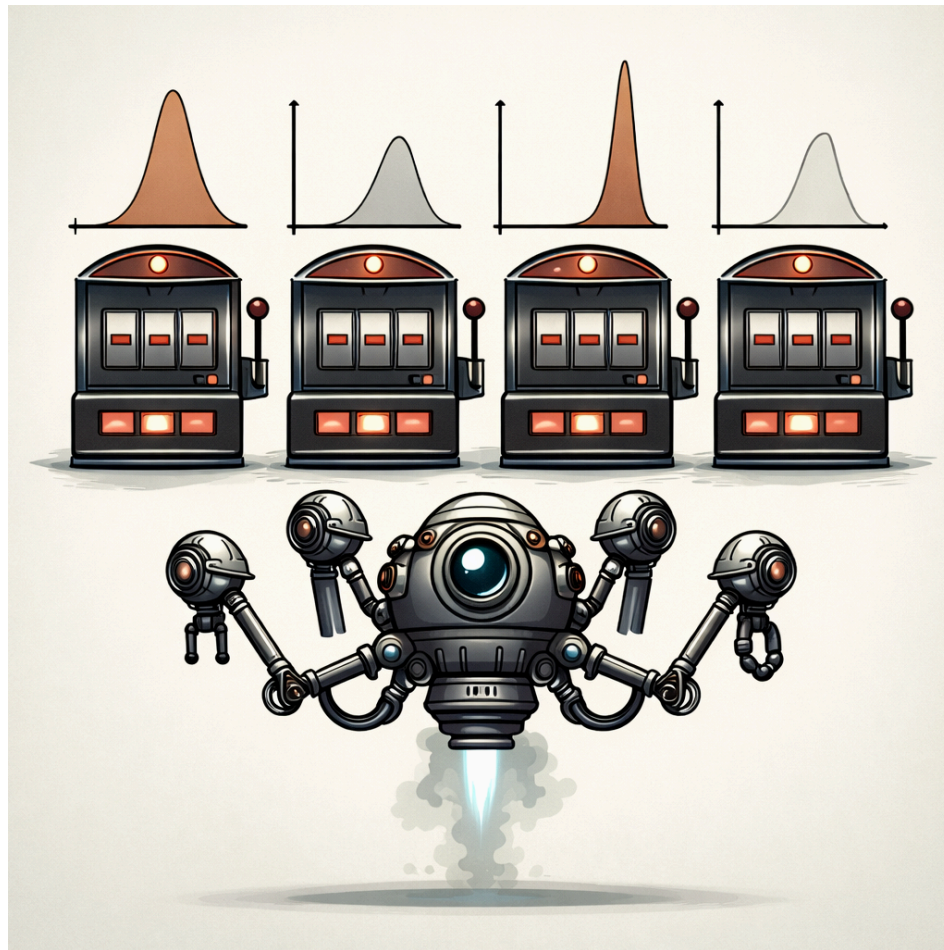
$$J_T = \mathbb{E} \left[\sum_{t=1}^T r_t \right]$$

Regret:

$$R_T = Tq^* - \mathbb{E} \left[\sum_{t=1}^T r_t \right]$$

where $q^* = \max_a \mathbb{E}[r \mid a]$.

Goal: sublinear regret $R_T = o(T)$.



k-Armed Bandit as a Minimal Model

At each step choose $a_t \in \{1, \dots, k\}$. Reward drawn from stationary distribution with mean $q(a)$.

Estimate:

$$Q_{t+1}(a) = Q_t(a) + \alpha(r_t - Q_t(a))$$

For sample average ($\alpha = \frac{1}{N_t(a)}$):

$$Q_t(a) = \frac{1}{N_t(a)} \sum_{i:a_i=a} r_i$$

Bandits isolate exploration problem: no state, no credit assignment.

Undirected Exploration: Epsilon-Greedy

With probability $1 - \varepsilon$:

$$a_t = \arg \max_a Q_{t(a)}$$

With probability ε :

$$a_t \sim \text{Uniform}(A)$$

Expected action probability:

$$P(a) = (1 - \varepsilon)1[a = a^*] + \frac{\varepsilon}{k}$$

[Optionally] Decaying schedule:

$$\varepsilon_t = \frac{c}{t}$$

Simple, works in practice

**Explores uniformly (ignores uncertainty).
Constant epsilon gives linear regret**

Softmax (Boltzmann) Exploration

Define stochastic policy:

$$\pi(a) = \frac{e^{\frac{Q(a)}{\tau}}}{\sum_b e^{\frac{Q(b)}{\tau}}} = \text{softmax}(A, a, \tau)$$

Temperature τ controls exploration:

- $\tau \rightarrow 0$: greedy
- $\tau \rightarrow \infty$: uniform

Gradient of log-policy (useful later):

$$\nabla_Q \log \pi(a) = \frac{1}{\tau} (1[a = a_t] - \pi(a))$$

Still uncertainty-unaware. But at least sensitive to the values. Unfortunately, also sensitive to scale of Q .

Optimism in the Face of Uncertainty

Idea: prefer actions with high uncertainty.

Optimistic initialization: Initialize $Q_{t=0}(a)$ large.

Until sampled enough times:

$r_t - Q_t(a) < 0 \Rightarrow$ value pushed down $\Rightarrow$ implicit exploration without randomness.

Limitation: disappears with constant step-size.

Upper Confidence Bound (UCB)

Principle: mean + uncertainty bonus.

$$a_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln(t)}{N_t(a)}} \right]$$

Hoeffding inequality (confidence interval):

$$P(|Q_t(a) - q(a)| > \varepsilon) \leq 2e^{-2N_t(a)\varepsilon^2}$$

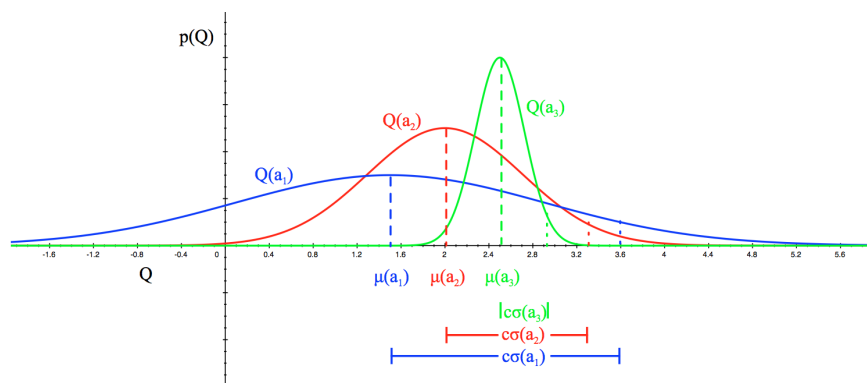
Confidence radius scales as:

$$\varepsilon \sim \sqrt{\frac{\ln(t)}{N_t(a)}}$$

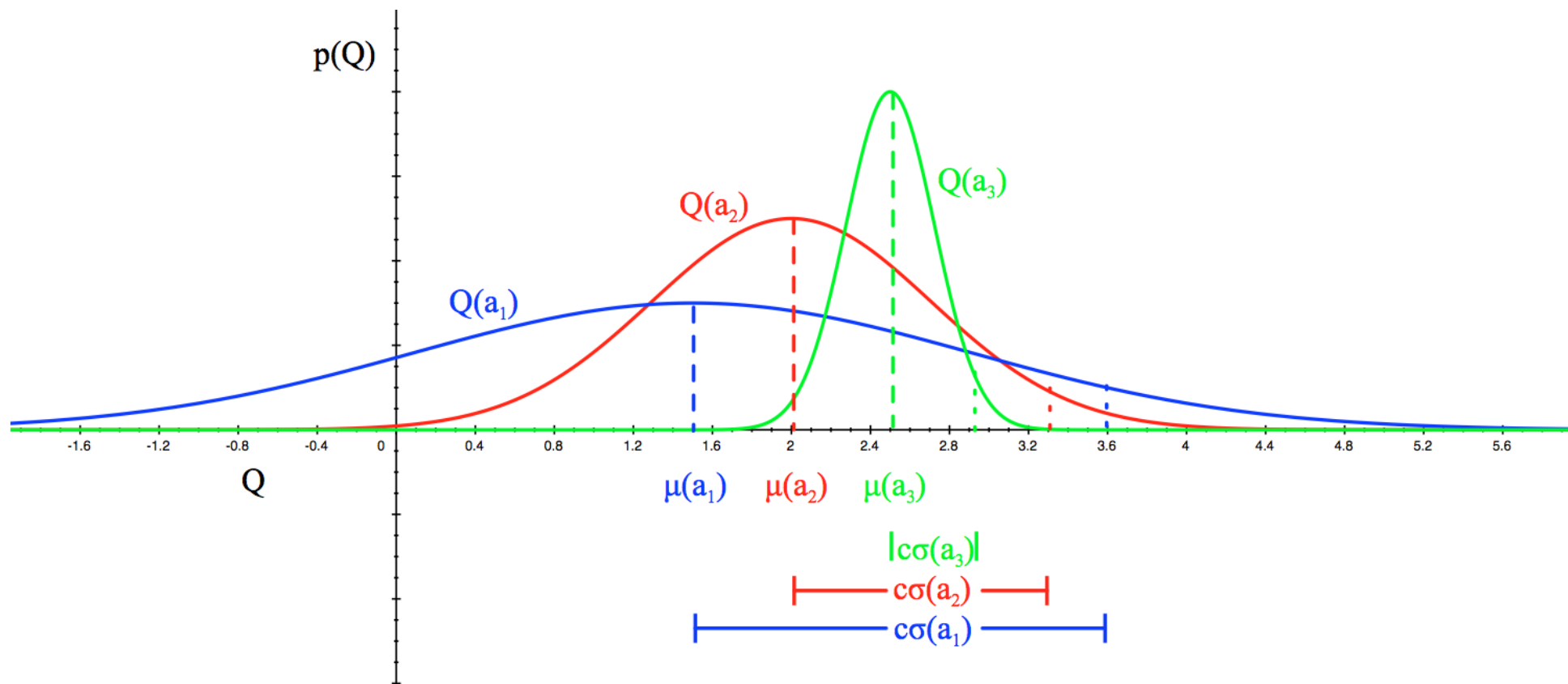
Interpretation:

- $Q_t(a)$ estimates mean
- bonus shrinks as $N_t(a)$ grows
- $\ln t$ ensures sufficient exploration

Leads to logarithmic regret $O(\ln T)$.



UCB illustration



Thompson Sampling (Bayesian View)

Maintain posterior over $q(a)$.

At step t :

- sample $\tilde{q}(a)$ from posterior
- choose $a_t = \arg \max_a \tilde{q}(a)$

Example: Bernoulli rewards (each arm).

- Prior: $B(\alpha, \beta)$.
- After S successes, F failures:
- Posterior: $B(\alpha + S, \beta + F)$.

Probability of selecting arm equals probability it is optimal under posterior.

Exploration proportional to uncertainty. Often empirically strong.

Comparing Exploration Strategies

Noise-based:

- epsilon-greedy
- softmax

Uncertainty-based:

- UCB
- Thompson

Key axes:

- directed vs undirected
- theoretical regret guarantees (i.e. $O(\cdot)$)
- hyperparameter sensitivity

Bandits: clear regret analysis.

MDPs: harder — exploration is about states, not only actions.

From Bandits to MDP

In MDP we optimize:

$$J = \mathbb{E} \left[\sum_t \gamma^t r_t \right]$$

ε -greedy in Q-learning:

$$a_t \sim \varepsilon\text{-greedy}(Q(s_t, \cdot))$$

Exploration affects state visitation distribution.

Tabular: sufficient exploration guarantees (GLIE, visiting all state-action pairs).

Deep RL: generalization + function approximation $\rightarrow$ local overfitting, state aliasing, bootstrapping bias.

Problem in Deep RL: Need sequences of correct exploratory actions. Action noise is often insufficient.

Intrinsic bonuses (will be discussed in a future lecture):

$$r'_t = r_t + \beta b(s_t), \text{ where } b(s) \text{ measures novelty / uncertainty.}$$

Exploration in Policy Methods

Policy gradient objective:

$$J(\theta) = \mathbb{E} \left[\sum_t \log \pi_{\theta}(a_t | s_t) G_t \right]$$

Entropy:

$$\mathbb{H}(\pi(s)) = - \sum_a \pi(a|s) \log \pi(a|s)$$

Stochastic policy already explores. But we can switch to entropy-regularized objective:

$$J_{\text{ent}}(\theta) = \mathbb{E} \left[\sum_t [\log \pi_{\theta}(a_t | s_t) A_t + \beta \mathbb{H}(\pi_{\theta}(s_t))] \right]$$

Gradient adds term:

$$\beta \nabla_{\theta} \mathbb{H}(\pi_{\theta}(s_t))$$

Encourages higher entropy policies. Controls exploration at objective level.

*Can be seen as KL-regularized control toward uniform prior. Equivalent to maximize $G(s) + \beta * \mathbb{E}[\mathbb{H}(s)]$*

Exploration and value estimates

Exploration quality depends on value estimates.

High-variance targets degrade learning.

Next: n-step returns, TD(λ), and advantage estimation.

Better return estimation $\rightarrow$ more stable improvement $\rightarrow$ more effective exploration.

Bias-Variance Tradeoff in Return Estimation

Credit assignment problem: how far into the future should we look?

Monte Carlo target:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}$$

- **unbiased (if episodes complete)**
- **high variance**

TD(0) target:

$$r_t + \gamma V(s_{t+1})$$

- **biased (bootstrapping)**
- **low variance**

Bootstrapping introduces bias, but reduces variance.

High-variance targets $\rightarrow$ unstable updates $\rightarrow$ unstable policy improvement.

Monte Carlo vs TD(0)

Value update (MC):

$$V(s_t) \leftarrow V(s_t) + \alpha[G_t - V(s_t)]$$

Value update (TD(0)):

$$V(s_t) \leftarrow V(s_t) + \alpha[r_t + \gamma V(s_{t+1}) - V(s_t)]$$

Define TD error:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

MC: no bootstrapping, episodic. TD: bootstrapping, online, step-wise.

MC propagates reward only after episode ends. TD propagates one step per update.

n-Step Returns

Idea: Interpolate between MC and TD — *take further bootstraps*:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V(s_{t+n})$$

Special cases:

- $n = 1 \Rightarrow \text{TD}(0)$
- $n \rightarrow \infty$ (episodic) $\Rightarrow \text{MC}$

TD(n) update:

$$V(s_t) \leftarrow V(s_t) + \alpha \left(G_t^{(n)} - V(s_t) \right)$$

Bias–variance interpolation via horizon n .

Larger $n \Rightarrow$ less bias, more variance, faster reward propagation. But longer time gap.

Forward View in Control

n-step SARSA target:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n Q(s_{t+n}, a_{t+n})$$

n-step Q-learning (bootstrapped on max):

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max_a Q(s_{t+n}, a)$$

Effect:

- accelerates reward propagation
- changes stability under function approximation

Choice of n becomes algorithmic design parameter.

TD(λ): Forward View

Idea: Interpolate between MC and TD — *combine all n -step returns*.

λ -return:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

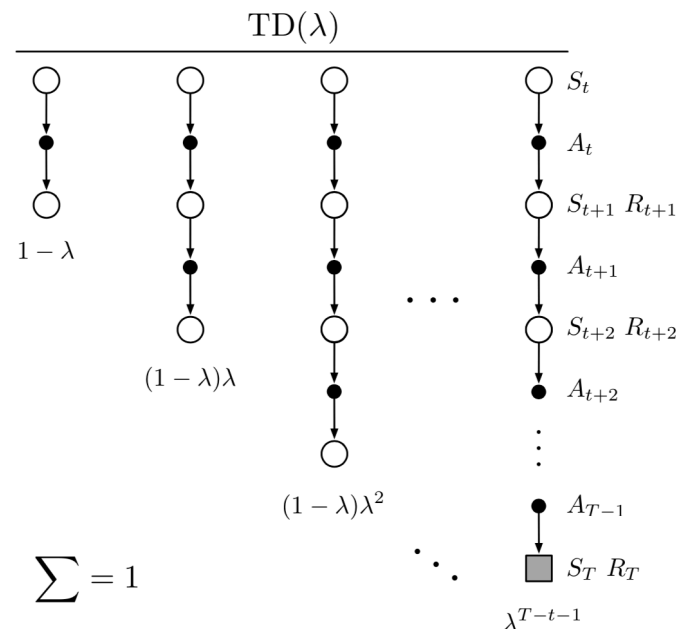
Exponential weighting over horizons.

Limits:

- $\lambda = 0 \rightarrow \text{TD}(0)$
- $\lambda = 1$ (episodic) $\rightarrow \text{MC}$

λ controls bias-variance tradeoff.

Smooth interpolation instead of fixed choice of n .



TD(λ): From n-step Mixture to TD-Errors

Start from definition:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)},$$

$$\text{where } G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V(s_{t+n})$$

Recall TD-error: $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$. Rearrange: $r_t = \delta_t + V(s_t) - \gamma V(s_{t+1})$

Expand mixture in G_t^λ and then substitute r_t :

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \left[\sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V(s_{t+n}) \right]$$

Geometric cancellation: $(1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} = (1 - \lambda) \frac{1}{1 - \lambda} = 1$

TD(λ): From n-step Mixture to TD-Errors (2)

After cancelling out terms, we obtain:

$$G_t^\lambda = V(s_t) + \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k},$$

where

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Key insight:

- TD(λ) $\rightarrow$ exponentially weighted accumulation of future TD-errors [/-residuals].
- Lambda controls how far TD-errors propagate backward in time.
- **offline calculation: wait for the end of an episode or some horizon T**

Backward View: Eligibility Traces

Idea: Instead of waiting for future rewards (offline), assign credit backward (online).

Eligibility trace (tabular):

$$e_t(s) = \gamma \lambda e_{t-1}(s) + 1[s_t = s]$$

TD error:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Update:

$$V(s) \leftarrow V(s) + \alpha \delta_t e_{t(s)}$$

Intuition:

- Recent states get large trace. Past states fade exponentially.
- For a fixed s , each timestep we apply exactly a single term from the prev slide equation.

Forward view = backward view *if applied offline*.

TD(λ) in Control

State-action traces:

$$e_{t(s,a)} = \gamma \lambda e_{t-1}(s,a) + 1[s_t = s, a_t = a]$$

Update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \delta_t e_{t(s,a)}$$

Variants:

- accumulating traces
- replacing traces

Practical impact:

- improves credit assignment
- may destabilize with nonlinear function approximation

The same idea applies to both SARSA and Q-learning. The only difference is in TD-error calculation.

From Value Learning to Advantage

Advantage definition:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

TD error as 1-step estimator:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Under on-policy sampling:

$$\begin{aligned}\mathbb{E}[\delta_t \mid s_t, a_t] &= r_t(s_t, a_t) + \gamma V(s_{t+1}) - V(s_t) \\ &= Q(s_t, a_t) - V(s_t) \\ &= A(s_t, a_t)\end{aligned}$$

Thus TD residual approximates advantage.

Generalized Advantage Estimation (GAE)

Note that previously we've coined λ -returns in TD(λ) defined via TD-errors.

Idea: since $A(s, a)$ can be expressed via TD-errors $\Rightarrow$ easy connection between TD(λ) and $A(s, t)$

GAE:

$$A_t^{\text{GAE}_{\gamma, \lambda}} = \lambda\text{-returns } G_t^\lambda - V(s_t) = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}$$

Policy gradient with GAE:

$$\nabla J(\theta) \approx \mathbb{E} [A_t^{\text{GAE}} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

Lambda controls bias-variance in policy gradient. Lower variance $\rightarrow$ more stable updates.

There are two main implementation of the GAE calculation:

- *original paper GAE formula: rightmost part of GAE eq via TD-errors*
- *calculate λ -returns first, then subtract $V(s)$: middle part of GAE eq*

Big Picture

MC $\leftrightarrow$ TD(0) unified via n-step.

n-step unified via lambda.

TD(λ) in value learning.

GAE in policy gradient.

And you can utilise those λ -returns in both actor and critic updates!

Core idea: Control bias–variance tradeoff in return estimation.

Better targets $\rightarrow$ lower variance gradients $\rightarrow$ more stable policy improvement.